





PREPARED BY

Umar Niaz  
Red Team Instructor at  
Cyberster

WEEK 1  
4 TASKS

- Target :-

hackthissite.org

# Advanced Recon & Attack Surface Mapping

- **Objective:** To master the art of information gathering and identifying the digital footprint of a target without being detected
- **Task 01 – Passive OSINT & Subdomain Enumeration**
- **Description:** Conduct a thorough passive reconnaissance on a given target domain (e.g., a bug bounty target or a dummy corporate domain).
- **Technical Requirements:**
  - Perform subdomain discovery using a combination of tools: **Subfinder**, **Assetfinder**, and **Amass**.
  - Use **DNS Dumpster** and **CRT.sh** to find historical subdomains and SSL certificate leaks.
  - Identify "hidden" assets that are not indexed by search engines.
- **Learning Goal:** Understand that no single tool is enough; cross-referencing results is key to a successful Red Team operation



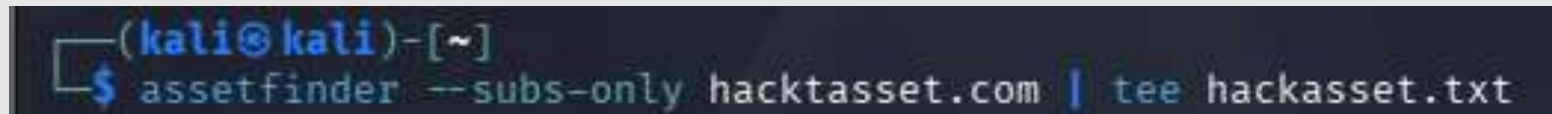
## Assetfinder

Used to discover subdomains associated with a target domain.

Queries multiple public sources like:

- Certificate Transparency logs
- Threat intelligence APIs
- DNS-related services

```
assetfinder --subs-only hackthissite.com | tee subs.txt
```



```
(kali@kali)-[~]  
$ assetfinder --subs-only hacktasset.com | tee hackasset.txt
```

```
assetfinder --subs-only hackthissite.com > assetfinder.txt
```

--subs-only

Outputs only subdomains of the target domain.

Prevents unrelated domains from polluting results.

Different data sources.

Used for cross-verification.

## **Amass (Passive)**

**Discovering and organizing all externally exposed assets of a target.**

**In simple terms: it helps us discover** domains, subdomains, IPs, ASN info, and infrastructure relationships **for a target organization.**

- · Servers
- · IP addresses
- · Cloud platforms
- · DNS servers
- · Load balancers
- · CDNs
- · Email servers
- · Databases
- · Hosting providers

**amass enum -passive -d target.com -o amass.txt**



```
(kali@kali)-[~]  
$ amass enum --passive -d hackthissite.org -o amasshack.txt  
hackthissite.org (FQDN) → mx_record → aspmx.l.google.com (FQDN)  
hackthissite.org (FQDN) → mx_record → alt1.aspmx.l.google.com (FQDN)  
hackthissite.org (FQDN) → mx_record → aspmx5.googlemail.com (FQDN)  
hackthissite.org (FQDN) → mx_record → aspmx3.googlemail.com (FQDN)  
hackthissite.org (FQDN) → mx_record → aspmx2.googlemail.com (FQDN)  
hackthissite.org (FQDN) → mx_record → aspmx4.googlemail.com (FQDN)  
hackthissite.org (FQDN) → mx_record → alt2.aspmx.l.google.com (FQDN)  
hackthissite.org (FQDN) → ns_record → c.ns.buddyns.com (FQDN)  
hackthissite.org (FQDN) → ns_record → h.ns.buddyns.com (FQDN)  
hackthissite.org (FQDN) → ns_record → g.ns.buddyns.com (FQDN)  
hackthissite.org (FQDN) → ns_record → j.ns.buddyns.com (FQDN)  
hackthissite.org (FQDN) → ns_record → f.ns.buddyns.com (FQDN)  
^C0-v3stage-cdn.hackthissite.org (FQDN) → cname_record → v3stage-cdn.hackthis  
site.org (FQDN)  
2-v3stage-cdn.hackthissite.org (FQDN) → cname_record → v3stage-cdn.hackthissi  
te.org (FQDN)
```

- Gather broader datasets.
- Often discovers additional domains.

## □ Why Amass Keeps Going(take longer time)

Even in --passive mode, Amass:

- Queries dozens of OSINT data sources
- Pulls certificate transparency logs
- Checks reverse DNS
- Resolves discovered subdomains
- Maps ASN + netblocks
- Recursively expands findings

For a large enterprise like hackthissite, that can mean:

- Thousands of historical records
- CDN expansions
- Multiple AWS accounts
- Delegated subdomains

It can run 10–30+ minutes depending on:

- Internet speed
- API limits
- Number of sources enabled
- Target size

- Merge Results
- `cat subfinder.txt assetfinder.txt amass.txt | sort -u > master_subdomains.txt`

- ```
(kali@kali)-[~]
$ cat subhack.txt assethack.txt amasshack.txt | sort -u > master_subdomains.txt
```

- `cat master_subdomains.txt`

```
(kali@kali)-[~]
$ cat master_subdomains.txt

0-v3stage-cdn.hackthissite.org (FQDN) → cname_record → v3stage-cdn.hackthissite.org (FQDN)
2-v3stage-cdn.hackthissite.org (FQDN) → cname_record → v3stage-cdn.hackthissite.org (FQDN)
hackthissite.org (FQDN) → mx_record → alt1.aspmx.l.google.com (FQDN)
hackthissite.org (FQDN) → mx_record → alt2.aspmx.l.google.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx2.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx3.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx4.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx5.googlemail.com (FQDN)
hackthissite.org (FQDN) → mx_record → aspmx.l.google.com (FQDN)
hackthissite.org (FQDN) → ns_record → c.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → f.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → g.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → h.ns.buddyns.com (FQDN)
hackthissite.org (FQDN) → ns_record → j.ns.buddyns.com (FQDN)
```

- `sort -u` Removes duplicates, Sorts entries alphabetically
- Creates single clean attack surface list. In totality we merged the results from our three tools ,subfinder, amass, assetfinder. Let's try more tools to get more subdomains
- **Count Unique Assets**
- `wc -l master_subdomains.txt`
- **wc → Counts lines, words, or bytes in a file.-l**
- **-l → Counts number of lines in a file.**  
**Shows total entries in your list.**

```
(kali㉿kali)-[~/Week1 Task1]  
$ wc -l master_subdomain.txt  
1034 master_subdomain.txt
```

here:

- **1034** = number of **lines** in the file
- **master\_subdomain.txt** = the file that was counted

There are **1034 lines** inside master\_subdomain.txt.

Shows real attack surface size.

Removes inflated numbers.

## Overlap Check by combining multiple subdomain files

**We will combine multiple subdomain files and process them to measure how often each subdomain appears across sources.**

```
sort subfinder.txt assetfinder.txt amass.txt | uniq -c | sort -nr | head
```

Higher repetition = higher confidence.

Single-source findings need validation.

### sort

Orders lines alphabetically.

Organizes data for processing.

- **uniq -c**

Counts duplicate occurrences of each line.

Shows how many times each subdomain appears.

- **sort -nr**  
Sorts numerically in descending order.  
Shows highest duplicate counts first.
- **head**  
Shows first 10 lines of output by default.  
Displays top results only.

```
(kali㉿kali)-[~]  
$ sort subhack.txt assethack.txt amasshack.txt | uniq -c | sort -nr | head
```

- We did this step to **prioritize and validate our recon data**.
- Combined multiple files, count repetition= Which assets are more likely real and important.

Part 2 Use **DNS Dumpster** and **CRT.sh** to find historical subdomains and SSL certificate leaks.

- Identify "hidden" assets that are not indexed by search engines.

crt.sh queries **Certificate Transparency logs** to extract:

- Subdomains
- Historical domains
- Staging environments
- Forgotten assets
- Wildcard certificates
- Internal naming leaks
- `crt_v2.sh -d hackthissite.com > crt_hack.txt`

DNS DUMPSTER:-

Go to:

<https://dnsdumpster.com>

Search:

target.com

Download / copy the **Subdomain list** → save as:

Dnsdumpster.xlsx

Then install in Kali linux

```
sudo apt install xlsx2csv -y
```

```
xlsx2csv -a robinhood.xlsx > all_sheets.csv
```

After Conversion

```
grep -oE "[a-zA-Z0-9.-]+\.\hackthssite\.org" all_sheets.csv | sort -u > dnsdump_robinhood.txt
```

- combine safely all:
- `cat dnsdump_robinhood.txt robinsub.txt robinasset.txt robinamass.txt crt_robin.txt | sort -u > combined.txt`
- OR
- `cat master_subdomain.txt dnsdumpster.txt crt_robin.txt | sort -u > attack_surface.txt`



# Why We Use DNS Dumpster

- Pulls DNS records
- Sometimes finds subdomains tools miss.  
Shows related IP infrastructure.

## Add To Master List

```
cat final_master.txt dnsdumpster.txt | sort -u > attack_surface.txt
```

Now this is your expanded dataset.

# Task 02 – Infrastructure & Tech Profiling

## WhatWeb

whatweb <https://sub.target.com>

- Web Server (Apache, Nginx, IIS)
- CMS (WordPress, Drupal, Joomla)
- Frameworks (Laravel, Django, React)
- Analytics & tracking tools
- CDN usage

## Why it matters:

If you detect:

- WordPress → Plugin vulnerabilities become relevant
- Laravel → Look for .env exposure
- Apache + outdated version → Known CVEs
- Cloudflare → Real IP discovery needed

## Wappalyzer CLI

wappalyzer --target <https://hackthissite.org/> --disable-ssl --output output.txt --json

**Confirms tech stack.** Verifies what technologies the website is actually using (e.g., server, framework, CDN, analytics).

**Cross-check fingerprint accuracy.** Double-checks that the detected technologies are correct by comparing multiple indicators (headers, scripts, responses) to avoid false positives.

## But if the wappalyzer doesn't give correct result use

Use Node Wappalyzer

Install Node (if not installed):

```
sudo apt install nodejs npm
```

Check version:

```
node -v
```

```
npm -v
```

Now run:

```
npx wappalyzer https://hackthissite.org
```

## You can also use chrome extension



[React](#)



[Next.js](#) 10.2.3

### Issue trackers



[Sentry](#)

### Security



[HSTS](#)

### Web frameworks



[Next.js](#) 10.2.3

### Miscellaneous



[Open Graph](#)

### Web servers



[Next.js](#) 10.2.3

### Development



[Emotion](#)

### Static site generators



[Next.js](#) 10.2.3

### JavaScript libraries



[Lodash](#) 4.17.21



[core-js](#) 2.6.12

### PaaS



[Amazon Web Services](#)

### Reverse proxies



[Envoy](#)

Something wrong or missing?

# Executive Summary Version

| Layer            | Technology             |
|------------------|------------------------|
| Frontend         | React + Next.js 10.2.3 |
| Hosting          | AWS                    |
| CDN              | CloudFront             |
| Reverse Proxy    | Envoy                  |
| Monitoring       | Sentry                 |
| Tracking         | LinkedIn Insight Tag   |
| Security Control | HSTS Enabled           |

## WHOIS

whois target.com

```
(kali@kali)-[~]  
$ whois hackthissite.com > whoishack.txt
```

Look for:

Registrar

Organization name

Name Servers

NetRange / CIDR blocks

Abuse contacts

- **Identifies ownership and ASN clues.** network (ASN)  
**Useful for infrastructure mapping.** how the target's servers and networks are structured and connected.

- **NSLookup**

`nslookup sub.target.com`

**Resolves IP and DNS info.** Finds the IP address and DNS records linked to the domain.

**Helps detect hosting patterns.** Shows where and how the domain is hosted (e.g., same IP for many subdomains, CDN usage, shared hosting).

CDN A network of distributed servers that deliver website content from locations closer to users.

```
(kali@kali)-[~]  
$ nslookup hackthissite.com  
Server:      192.168.1.1  
Address:     192.168.1.1#53  
  
Non-authoritative answer:  
Name:   hackthissite.com  
Address: 13.248.169.48  
Name:   hackthissite.com  
Address: 76.223.54.146
```

```
(kali@kali)-[~]  
$ nslookup hackthissite.com > hackns.txt  
  
—(kali@kali)-[~]
```

## Task 2 Part 2

Perform **advanced Google Dorking** to find sensitive files like phpinfo(), .env, robots.txt, and backup directories (.bak, .zip).

### Finding phpinfo() Pages

#### Search in Google

site:target.com inurl:phpinfo.php

site:target.com intitle:"phpinfo()"

Why this matters:

Reveals PHP version

Shows loaded modules

Displays server paths

Environment variables



## Query Specific Records

A Record

```
nslookup -type=A target.com
```

MX Record

```
nslookup -type=MX target.com
```

NS Record

```
nslookup -type=NS target.com
```

TXT Record

```
nslookup -type=TXT target.com
```

🔥 Alternative (More Powerful) — dig

```
dig target.com ANY
```

Or:

```
dig target.com MX
```

```
dig target.com NS
```

```
dig target.com TXT
```

📡 What You're Actually Mapping (Creating a structured overview target's infrastructure)

You're building:

Hosting provider identification

Cloud provider detection (AWS / Azure / GCP)

CDN detection (CloudFront / Akamai / Cloudflare)

Email infrastructure mapping

Subdomain discovery hints

## 🔗 Finding .env Files (Critical)

site:target.com filetype:env

site:target.com inurl:.env

These may expose:

Database credentials

API keys

Mail server passwords

Secret tokens

⚠️ Most real-world breaches start from exposed .env.

## 🔗 Finding robots.txt

site:target.com inurl:robots.txt

Robots often reveal:

Admin panels

Private directories

Staging environments

API paths

Always manually inspect it.

## 🔗 Finding Backup Files (.bak, .zip, .old, .tar)

site:target.com filetype:bak

site:target.com filetype:zip

site:target.com inurl:backup

site:target.com ext:sql

site:target.com ext:old

Backup archives sometimes contain:

Full source code

SQL dumps

Configuration files

## 🔗 Directory Listing Discovery

site:target.com intitle:"index of"

If directory listing is enabled, you may find:

Uploaded files

Logs

Backups

Internal documents

- **Task 03: GitHub Dorking & Credential Leaks**

- **Description:** Search for leaked secrets and hardcoded credentials in public repositories.
- Practical Demonstration – Manual GitHub Search
- Go to: <https://github.com>
- Use the search bar at the top

## Basic Organization-Based Search

### Example

org:targetorg "API\_KEY"

org:targetorg "PASSWORD"

org:targetorg "AWS\_SECRET"

org:targetorg "SQL\_CONNECTION"

## Broad Public Search (Outside Org Scope)

"target.com" "API\_KEY"

"targetorg" "AWS\_SECRET\_ACCESS\_KEY"

"targetorg" "PRIVATE\_KEY"

Sometimes employees push code to personal repos instead of official org repos.

## Use Advanced Search Filters

After searching:

- Click **Code** (not Issues, not Users)
- Filter by: Recently updated
  - Language

Better structured query:

org:target "API\_KEY"

Or:

"target.com" AND "API\_KEY"

## File-Specific Searches

filename:.env "target"

filename:config "DB\_PASSWORD"

filename:settings.py "SECRET\_KEY"

extension:json "apiKey"

Common risky files:

.env

config.js

settings.py

application.properties

web.config

## What to Look For

You are NOT hunting for exploitation.

You are checking:

- Is the value hardcoded?
- Is it real-looking or test?
- Is it commented out?
- Is it part of documentation?

Example risky patterns:

```
const API_KEY = "sk_live_93ksk29..."
```

```
AWS_SECRET=AKIAIOSFODNN7...
```

```
password = "Admin@123"
```

## Commit History Analysis (Why Deleting Is Not Enough)

### ◦ If a repository within is public:

- Open the file.
- Click **History** (top right).
- Browse older commits.
- “Even if a secret was removed later, Git preserves previous versions.”
- Look for:
  - Added credentials in older commits
  - Secrets later removed
  - Config files changed

### Using GitHacker (Authorized Repos Only)

```
githacker --url <URL_TO_.GIT_FOLDER> --output-folder <OUTPUT_DIRECTORY>
```

OR

```
githacker https://target.com/.git/ ./dumped_repo
```

After dumping:

```
cd dumped_repo
```

```
git log
```

This allows you to inspect commit history.

## Analyzing Commit History for Deleted Secrets.

Developers often:

Commit credentials

Realize mistake

Delete them in next commit

But Git preserves full history.

### 🔍 Check Full History

`git log -p`

🔍 Search Entire History for Keywords

`git grep -i "password" $(git rev-list --all)`

Or:

`git log -S "API_KEY" --all`

This shows commits where the string was added or removed.



## Task 04 – Live Asset Filtering & Visual Recon

**Description:** Filter out dead assets and visualize the live attack surface

- **Technical Requirements:**
  - Use **Httpprobe** or **Httpx** to verify which subdomains identified in Task 01 are actually alive.
  - Capture screenshots of all live web assets using **Aquatone** or **Gowitness** to identify interesting login panels or dev environments.
- **Learning Goal:** Save time by focusing only on active, high-value targets.

In Task 01 you probably collected:

- 200 subdomains
- 500 subdomains
- Maybe 1000+

But here's reality:

- ☞ Many are dead
- ☞ Many don't respond
- ☞ Many are parked
- ☞ Many are useless

If you try to test everything manually, you waste hours.

So Task 04 = Filter → Focus → Visualize

Take that merged file of subdomains enumeration (subdomains.txt) of Task 1 and  
cat subdomains.txt | httpprobe --prefer-https > live-urls.txt

- This will:
- Test both HTTP and HTTPS for each subdomain.
- Output only those that respond.
- Prefer HTTPS (if available) over HTTP.

- **Check the results:**

cat live-urls.txt

cat subdomains.txt | httpx -status-code -title -tech-detect -o live-httpx.txt

Capture Screenshots with Aquatone or Gowitness

cat live-urls.txt | aquatone -out ~/aquatone\_results

After the scan finishes, open the generated HTML report:

```
firefox ~/aquatone_results/aquatone_report.html
```

You'll see all screenshots in a grid – perfect for spotting login panels or dev environments.

### Option B: Using Gowitness

```
gowitness file -f live-urls.txt --screenshot-path  
~/gowitness_screenshots
```

To view the results, start the built-in web server:

```
gowitness report serve
```

Then open your browser to <http://127.0.0.1:7171>. You'll get a clean dashboard with all screenshots.

## What Goes in the Report

After filtering + screenshots, you document:

- Total subdomains found
- Total live hosts
- Interesting panels discovered
- Technologies observed
- Any obvious exposure (like open directories)

No exploitation.

Just surface mapping.

Task 04 is literally:

1. Remove dead domains.
2. Screenshot what's alive.
3. Focus on valuable assets.

🎯 Learning Goal

“Recon is not about collecting everything.

Recon is about reducing noise and identifying high-value active targets.”

That's it.

## Document Your Findings

For your assignment, you'll likely need to document:

Which subdomains were live.

Screenshots of interesting assets.

Observations about potential security risks (e.g., exposed admin panels).

You can save relevant screenshots from the Aquatone folder or take additional manual screenshots using tools like flameshot or gnome-screenshot.

Task Submission link

[https://docs.google.com/forms/d/e/1FAIpQLSeQ\\_5KG0oNxEKkBSzmo9cgsFdybe0NLPiSdxLU9F2jb1KHYbg/viewform](https://docs.google.com/forms/d/e/1FAIpQLSeQ_5KG0oNxEKkBSzmo9cgsFdybe0NLPiSdxLU9F2jb1KHYbg/viewform)



THANK YOU